

Spike: Task 4**Title:** GridWorld Multi-threaded**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike extension task is to demonstrate my ability to develop multitasking using the gridworld to show understanding of threading and non-blocking game loop.

Besides this report, what else was created?

- Code files in see COS30031-2023-102608927\04 - Spike - Gridworld Multi-threaded\Multi-Thread\Multi-Thread

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- Lecture 2.1 – Game Loops & Architecture

Tasks undertaken:

- Create new Project Solutions
- Import Task 3 GridWorld.cpp code into Task 4 folder.
- Implement Game Loop
- Implement Input Processing
- Implement Treading
- Use of mutex, atomic and condition_variable for Synchronization

Extended Implementation of Game Loop:

```
// Function to handles the game logic
void gameLoopThreadFunction(char grid[Grid_Size][Grid_Size])
{
    while (!isGameOver)
    {
        unique_lock<mutex> lock(mtx);
        cv.wait(lock, []
                { return inputReceived || isGameOver; }); // Waits for input or game over

        if (inputReceived)
        {
            // Process the input
            switch (user_cmd)
            {

```

We extend our previous game loop to run thread within our **gameLoopThreadFunction**. This loop operates continuously and is designed to handle game logic, when necessary, by utilizing a condition variable the loop waits, for user input before proceeding with any updates to the game state ensuring that input processing doesn't block operations. When the user provides input the game loop and then applies their command to their position in the game grid. Additionally, the loop checks if player won or lost and displays a message based on conditions such as finding gold or falling into a pit. This approach aligns with standards for blocking game loops as it efficiently utilizes system resources by pausing when there's no input, to process.

Extended Implementation of Input Processing:

```
// Function to handles user input
void inputThreadFunction()
{
    char input;
    while (!isGameOver)
    {
        while (clearingScreen)
        {
            this_thread::sleep_for(chrono::milliseconds(1));
        }
        // Lock to synchronize access to the console
        lock_guard<mutex> lock(mtx);
        cout << "Enter Your Command: ";

        cin >> input;
        if (!cin)
        {
            // Handles input error
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            {
                lock_guard<mutex> lock(mtx);
                cout << "Invalid command. Please use N, S, E, W, or Q." << endl;
            }
            continue;
        }

        input = toupper(input);
        // Handles valid commands and notify the game loop
        if (input == 'N' || input == 'S' || input == 'E' || input == 'W' || input == 'Q')
        {
            user_cmd = input;
            inputReceived = true;
            cv.notify_one();
        }
        else
        {
            // Handles invalid command
            lock_guard<mutex> lock(mtx);
            cout << "Invalid command. Please use N, S, E, W, or Q." << endl;
        }
    }
}
```

Furthermore, I had to make changes to our Input processing to be handled by a separate thread, **inputThreadFunction**, this ensures that the game loop remains non-blocking. User commands are read from the console and immediately reflected in an atomic variable **user_cmd**, signaling the game loop to wake up and process the input. This design allows for responsive input handling while decoupling it from the core game logic. The use of atomics and mutexes in this context is a standard practice for thread-safe communication between threads, which makes sure that race conditions are avoided, and that input is processed reliably.

Implementation Use of Thread in GridWorld:

Use of thread was a very important step in implementing Multi-threading in GridWorld and we do this by creating an asynchronous experience by what was already explain in running separate threads in the game logic and input handling where the program can execute the non-blocking game loop and process user input concurrently by using the standard libraries

```
#include <iostream>
#include <thread>
#include <mutex>
#include <atomic>
#include <chrono>
#include <limits>
#include <condition_variable>
```

such as **std::thread**, **std::mutex**, **std::condition_variable** and **std::atomic** as best practices to ensure thread synchronization and data integrity. The game loop thread will wait for a user input before proceeding to minimize CPU usage while in idle.

Open issues/risks

An issue that I came across was the display of “Enter Your Command:” where it is prompted only when the program is executed but after that is cleared from the screen and flashes for a split second on each loop. I believe that the issue lies within the clearscreen condition we use but due to time constraints I could not fix it.

```
#####
#G D#D #
#      #
###  # #
#      #
#   ## #
#      #
##P#####
Enter Your Command: _
```

```
#####
#G D#D #
#      #
###  # #
#      #
#   ## #
# P    #
##S#####
_
```